

Exercise 2: E-Commerce Platform Search Function

Solution

Big O Notation

Big O notation represents how the execution time or memory usage of an algorithm grows as the input size increases. It helps compare the efficiency of different algorithms and determine how well they perform for small and large datasets.

Search Complexity

Case	Linear Search	Binary Search
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$

```
import java.util.*;

class ProductInfo {
    int productId;
    String productName;
    String category;

    ProductInfo(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    void showDetails() {
        System.out.println("ID : " + productId);
        System.out.println("Name : " + productName);
        System.out.println("Category : " + category);
    }
}
```

```
public class EcommerceSearch {

    static ProductInfo linearFind(ProductInfo[] productList, int targetId) {
        for (ProductInfo product : productList) {
            if (product.productId == targetId) {
                return product;
            }
        }
        return null;
    }

    static ProductInfo binaryFind(ProductInfo[] sortedList, int targetId) {

        int start = 0;
        int end = sortedList.length - 1;

        while (start <= end) {

            int middle = start + (end - start) / 2;

            if (sortedList[middle].productId == targetId)
                return sortedList[middle];

            if (sortedList[middle].productId < targetId)
                start = middle + 1;
            else
                end = middle - 1;
        }

        return null;
    }
}
```

```
}
```

```
public static void main(String[] args) {
```

```
    Scanner sc = new Scanner(System.in);
```

```
    System.out.println("==== Product Search =====");
```

```
    System.out.print("Enter total products: ");
```

```
    int size = sc.nextInt();
```

```
    sc.nextLine();
```

```
    ProductInfo[] productList = new ProductInfo[size];
```

```
    for (int i = 0; i < size; i++) {
```

```
        System.out.println("\nEnter Product " + (i + 1));
```

```
        System.out.print("Product ID: ");
```

```
        int id = sc.nextInt();
```

```
        sc.nextLine();
```

```
        System.out.print("Product Name: ");
```

```
        String name = sc.nextLine();
```

```
        System.out.print("Category: ");
```

```
        String category = sc.nextLine();
```

```
        productList[i] = new ProductInfo(id, name, category);
```

```
    }
```

```
System.out.print("\nSearch Product ID: ");
int searchId = sc.nextInt();

ProductInfo linearResult = linearFind(productList, searchId);

System.out.println("\n--- Linear Search ---");

if (linearResult != null)
    linearResult.showDetails();
else
    System.out.println("No matching product found.");

ProductInfo[] sortedProducts = Arrays.copyOf(productList, productList.length);

Arrays.sort(sortedProducts, Comparator.comparingInt(p -> p.productId));

ProductInfo binaryResult = binaryFind(sortedProducts, searchId);

System.out.println("\n--- Binary Search ---");

if (binaryResult != null)
    binaryResult.showDetails();
else
    System.out.println("No matching product found.");

sc.close();
}
}
```

Output:

```
===== Product Search =====
Enter total products: 2

Enter Product 1
Product ID: 101
Product Name: laptop
Category: electronics

Enter Product 2
Product ID: 205
Product Name: shoes
Category: footwear

Search Product ID: 205

--- Linear Search ---
ID      : 205
Name     : shoes
Category : footwear

--- Binary Search ---
ID      : 205
Name     : shoes
Category : footwear
```

Analysis:

Binary Search performs much better than Linear Search when the data is arranged in sorted order. Linear Search checks elements one by one, resulting in $O(n)$ time in the average and worst cases. Binary Search repeatedly divides the search space into two halves, achieving $O(\log n)$ time complexity. For large e-commerce product collections, Binary Search is generally the better choice, provided the products are sorted before searching.

Exercise 7: Financial Forecasting

Solution

Recursion

Recursion is a programming technique where a method repeatedly calls itself to solve a problem by breaking it into smaller subproblems. The recursive calls continue until a base condition is reached, after which the results are returned back through the call stack

```
import java.util.Scanner;

public class FinancialForecast {

    static double predictValue(double amount, double interestRate, int years) {

        if (years == 0) {
            return amount;
        }

        return predictValue(
            amount * (1 + interestRate / 100),
            interestRate,
            years - 1
        );
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("===== Financial Forecast =====");

        System.out.print("Enter Initial Investment: ");

        double initialAmount = sc.nextDouble();

        System.out.print("Enter Annual Growth Rate (%): ");
```

```

double growthRate = sc.nextDouble();

System.out.print("Enter Forecast Period (Years): ");

int totalYears = sc.nextInt();

double futureAmount = predictValue(initialAmount, growthRate, totalYears);

System.out.printf("\nPredicted Value after %d years = %.2f\n",
    totalYears,
    futureAmount);

sc.close();
}
}

```

Output:

```

===== Financial Forecast =====
Enter Initial Investment: 10000
Enter Annual Growth Rate (%): 8
Enter Forecast Period (Years): 5

Predicted Value after 5 years = 14693.28

```

Analysis:

The recursive relation for this solution is $T(n) = T(n - 1) + O(1)$ because each recursive call performs only a constant amount of computation before invoking the next call. As a result, the overall time complexity is $O(n)$.

Although recursion makes the implementation simple and easy to understand, it introduces additional function call overhead and uses extra stack memory. An iterative solution using a loop, or techniques such as memoization when solving overlapping subproblems, can improve efficiency and reduce memory consumption.